

Cinemática Directa e Inversa del Robot KUKA KR 7 R900-3 con ROS 2 Jazzy

Manuel Zapana Adrian Pacheco

IMT-342 Robótica — Universidad Católica Boliviana “San Pablo”

Docente: Carlos Daniel Aguilar Mancachi

Resumen—Este informe presenta el modelado y la implementación de la cinemática directa e inversa de posición para el manipulador KUKA KR 7 R900-3 de seis grados de libertad bajo ROS 2 Jazzy. Se realizó una asignación de marcos y una formulación mediante la convención Denavit–Hartenberg estándar, incorporando la transformación fija utilizada por el modelo implementado. La cinemática directa obtiene la posición y orientación del efector final. Para la cinemática inversa se implementó un método iterativo con Jacobiano posicional numérico, pseudoinversa amortiguada, límites articulares y múltiples semillas. Los resultados fueron validados comparando la FK analítica con las transformaciones TF publicadas por ROS 2 y verificando tres objetivos cartesianos mediante RViz2.

Index Terms—Denavit-Hartenberg, cinemática directa, cinemática inversa, Jacobiano, ROS 2, RViz2, KUKA KR 7 R900-3.

I. DESCRIPCIÓN DEL ROBOT ASIGNADO

El robot utilizado es el manipulador industrial KUKA KR 7 R900-3, de seis articulaciones de revolución. En el modelo ROS 2 las articulaciones se identifican como `joint_1` a `joint_6`. La IK implementada resuelve únicamente posición cartesiana, mientras que la FK permite obtener tanto posición como orientación. De acuerdo con la documentación de producto de KUKA, el KR 7 R900-3 pertenece a la familia KR AGILUS y presenta un alcance máximo de 902 mm y una carga nominal de 7 kg.[1]

Cuadro I
DATOS GENERALES DEL KUKA KR 7 R900-3

Fabricante	KUKA
Modelo	KR 7 R900-3
Grados de libertad	6
Tipo de articulaciones	Revolutas
Alcance máximo	0.902 m
Carga nominal	7 kg
Marco base	<code>base_link</code>
Efector final	<code>tool0</code>
Unidades	m y rad

II. ASIGNACIÓN DE MARCOS DE REFERENCIA Y PARÁMETROS DH

Se empleó la convención Denavit–Hartenberg estándar. La matriz elemental utilizada por los nodos es

$$A_{i-1}^i = R_z(\theta_i)T_z(d_i)T_x(a_i)R_x(\alpha_i), \quad (1)$$

con

Cuadro II
LÍMITES ARTICULARES UTILIZADOS

Articulación	Mínimo [rad]	Máximo [rad]
q_1	-3,22886	3,22886
q_2	-3,31613	0,69813
q_3	-2,00713	2,79253
q_4	-3,49066	3,49066
q_5	-2,09440	2,09440
q_6	-6,10865	6,10865

$$A_{i-1}^i = \begin{bmatrix} c_\theta & -s_\theta c_\alpha & s_\theta s_\alpha & ac_\theta \\ s_\theta & c_\theta c_\alpha & -c_\theta s_\alpha & as_\theta \\ 0 & s_\alpha & c_\alpha & d \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (2)$$

La asignación de frames se realizó considerando que cada eje Z coincide con el eje de movimiento de su articulación, mientras que los ejes X se establecen de acuerdo con la relación geométrica entre ejes Z consecutivos. El eje Y se obtiene mediante la regla de la mano derecha.

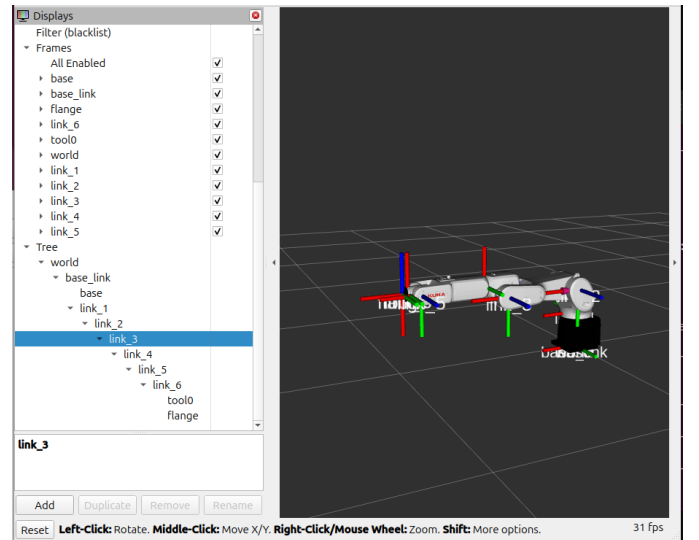


Figura 1. Asignación y árbol de marcos de referencia observados en RViz2.

La Figura 1 muestra la cadena de frames del modelo ROS 2, desde `base_link` hasta `link_6`, junto con `flange` y `tool0`. La Figura 2 identifica las seis variables articulares q_1 – q_6 empleadas en la formulación. Los frames matemáticos utilizados para la formulación DH no tienen que coincidir

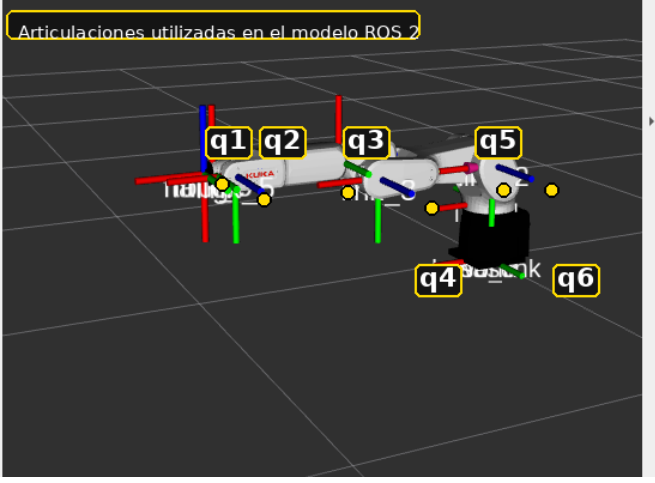


Figura 2. Identificación de las seis articulaciones utilizadas por la cadena cinemática en el modelo ROS 2.

literalmente con los nombres del URDF; la correspondencia se valida mediante TF.

Cuadro III
PARÁMETROS DH UTILIZADOS POR FK E IK

i	θ_i [rad]	d_i [m]	a_i [m]	α_i [rad]
1	q_1	-0,34200	0,05000	$+\pi/2$
2	q_2	0,09305	0,41000	0
3	$q_3 - \pi/2$	-0,09305	0,04500	$+\pi/2$
4	q_4	-0,44000	0	$-\pi/2$
5	q_5	0	0	$+\pi/2$
6	$q_6 + \pi$	-0,07700	0	π

Además, el código utiliza la transformación fija

$$T_b = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (3)$$

por lo que la FK implementada es

$$T_{base}^{tool0} = T_b A_1 A_2 A_3 A_4 A_5 A_6. \quad (4)$$

El frame `tool0` está unido de forma fija al extremo del robot y no constituye una séptima articulación. La transformación medida entre `link_6` y `tool0` fue

$${}^{link_6}T_{tool0} \approx \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & -0,026 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (5)$$

No se incorporó una herramienta física adicional ni un payload porque la práctica realizada exige alcanzar la posición cartesiana de `tool0`; el modelo utilizado no requiere introducir masa o dimensiones de una herramienta externa.

III. CINEMÁTICA DIRECTA

III-A. Matrices homogéneas particulares

Definiendo $c_i = \cos q_i$ y $s_i = \sin q_i$, las matrices que utiliza el código son

$$A_1 = \begin{bmatrix} c_1 & 0 & s_1 & 0,05c_1 \\ s_1 & 0 & -c_1 & 0,05s_1 \\ 0 & 1 & 0 & -0,342 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (6)$$

$$A_2 = \begin{bmatrix} c_2 & -s_2 & 0 & 0,41c_2 \\ s_2 & c_2 & 0 & 0,41s_2 \\ 0 & 0 & 1 & 0,09305 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (7)$$

$$A_3 = \begin{bmatrix} s_3 & 0 & -c_3 & 0,045s_3 \\ -c_3 & 0 & -s_3 & -0,045c_3 \\ 0 & 1 & 0 & -0,09305 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (8)$$

$$A_4 = \begin{bmatrix} c_4 & 0 & -s_4 & 0 \\ s_4 & 0 & c_4 & 0 \\ 0 & -1 & 0 & -0,440 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (9)$$

$$A_5 = \begin{bmatrix} c_5 & 0 & s_5 & 0 \\ s_5 & 0 & -c_5 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (10)$$

$$A_6 = \begin{bmatrix} -c_6 & -s_6 & 0 & 0 \\ -s_6 & c_6 & 0 & 0 \\ 0 & 0 & -1 & -0,077 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (11)$$

Por tanto,

$$T_{base}^{tool0} = \begin{bmatrix} R(q) & p(q) \\ 0_{1 \times 3} & 1 \end{bmatrix}. \quad (12)$$

Para hacer explícita la posición obtenida del modelo, se definen $c_{23} = \cos(q_2 + q_3)$ y $s_{23} = \sin(q_2 + q_3)$. Entonces

$$\begin{aligned} p_x = & -0,077(s_1 s_4 + c_1 c_4 s_{23}) s_5 \\ & + c_1(0,045 s_{23} + 0,410 c_2 + 0,440 c_{23} + 0,050) \\ & + 0,077 c_1 c_5 c_{23}, \end{aligned} \quad (13)$$

$$\begin{aligned} p_y = & 0,077(s_1 s_{23} c_4 - c_1 s_4) s_5 \\ & - s_1(0,045 s_{23} + 0,410 c_2 + 0,440 c_{23} + 0,050) \\ & - 0,077 s_1 c_5 c_{23}, \end{aligned} \quad (14)$$

$$\begin{aligned} p_z = & -0,410 s_2 - 0,077 s_5 c_4 c_{23} - 0,077 s_{23} c_5 \\ & - 0,440 s_{23} + 0,045 c_{23} + 0,342. \end{aligned} \quad (15)$$

La orientación corresponde al bloque $R(q)$ de T . De forma compacta puede expresarse como

$$R(q) = D R_1 R_2 R_3 R_4 R_5 R_6, \quad D = \text{diag}(1, -1, -1), \quad (16)$$

donde

$$R_1 = \begin{bmatrix} c_1 & 0 & s_1 \\ s_1 & 0 & -c_1 \\ 0 & 1 & 0 \end{bmatrix}, \quad R_2 = \begin{bmatrix} c_2 & -s_2 & 0 \\ s_2 & c_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad (17)$$

$$R_3 = \begin{bmatrix} s_3 & 0 & -c_3 \\ -c_3 & 0 & -s_3 \\ 0 & 1 & 0 \end{bmatrix}, \quad R_4 = \begin{bmatrix} c_4 & 0 & -s_4 \\ s_4 & 0 & c_4 \\ 0 & -1 & 0 \end{bmatrix}, \quad (18)$$

$$R_5 = \begin{bmatrix} c_5 & 0 & s_5 \\ s_5 & 0 & -c_5 \\ 0 & 1 & 0 \end{bmatrix}, \quad R_6 = \begin{bmatrix} -c_6 & -s_6 & 0 \\ -s_6 & c_6 & 0 \\ 0 & 0 & -1 \end{bmatrix}. \quad (19)$$

La matriz $R(q)$ se transforma a cuaternión para reportar la orientación del efector final. Por ejemplo, para la tercera configuración validada se obtuvo aproximadamente $[x, y, z, w] = [0,1433, 0,9496, -0,2788, -0,0041]$.

IV. JACOBIANO POSICIONAL

La relación diferencial de posición es

$$\dot{p} = J(q)\dot{q}, \quad J(q) \in \mathbb{R}^{3 \times 6}. \quad (20)$$

Aunque el Jacobiano puede derivarse analíticamente a partir de $p(q)$, el nodo implementado utiliza diferencias finitas. Para cada articulación se calcula

$$J_{:,i} = \frac{p(q + \delta e_i) - p(q)}{\delta}, \quad \delta = 10^{-6}, \quad (21)$$

donde e_i es el vector unitario asociado a la articulación i . Así, las seis columnas se obtienen directamente a partir de la misma FK utilizada para resolver la IK.

Este procedimiento mantiene consistente el Jacobiano con el modelo cinemático implementado y evita utilizar una expresión analítica diferente de la FK programada.

V. CINEMÁTICA INVERSA NUMÉRICA

El nodo `ik_node.py` recibe un punto cartesiano mediante `/target (geometry_msgs/msg/Point)` y busca una configuración q^* tal que

$$p(q^*) \approx p_d. \quad (22)$$

En cada iteración se calcula

$$e_k = p_d - p(q_k). \quad (23)$$

Para mejorar el comportamiento cerca de singularidades se utiliza la pseudoinversa amortiguada

$$J^\# = J^T(JJ^T + \lambda^2 I)^{-1}, \quad \lambda = 0,02. \quad (24)$$

El incremento articular implementado es

$$\Delta q = 0,6 J^\# e_k. \quad (25)$$

Si alguna componente supera 0,20 rad, todo el incremento se escala para cumplir

$$\max_i |\Delta q_i| \leq 0,20 \text{ rad}. \quad (26)$$

Posteriormente se actualiza

$$q_{k+1} = q_k + \Delta q \quad (27)$$

y se aplica saturación componente a componente usando los límites de la Tabla II.

El algoritmo acepta una solución cuando

$$\|e_k\|_2 < 10^{-3} \text{ m}, \quad (28)$$

con un máximo de 300 iteraciones por semilla.

V-A. Múltiples semillas

Se prueban seis configuraciones iniciales. La primera corresponde a la configuración actual obtenida de `/joint_states`; las otras cinco son configuraciones alternativas predefinidas. Todas se saturan a los límites articulares antes de comenzar la iteración.

Para cada semilla se calcula su error final. La solución seleccionada es la que presenta el menor error. Una solución solo se publica si además cumple la tolerancia de convergencia.

Cuando existe convergencia, el nodo actualiza `q_current`, activa la publicación de la solución y publica el nuevo vector de `/joint_states` mediante un temporizador de 0,05 s, equivalente a 20 Hz.

VI. ESTRUCTURA DEL WORKSPACE Y EJECUCIÓN EN ROS 2

El proyecto se organizó en el workspace `~/grupo2_kuka`. El paquete de cinemática se encuentra en `src/grupo02_robot_kinematics/`, dentro del módulo `grupo02_robot_kinematics/`. Los archivos principales son `fk_node.py`, `ik_node.py` y `__init__.py`.

VI-A. Obtención e instalación desde cero

Para reproducir el proyecto en una computadora nueva, primero se debe disponer de Ubuntu 24.04 con ROS 2 Jazzy instalado y de Git. El código no se copia manualmente: se obtiene directamente desde el repositorio del proyecto.

```
cd ~
git clone https://github.com/ManuelZapanaGutierrez/grupo2_kuka.git
cd ~/grupo2_kuka
```

Después de clonar el repositorio, se puede comprobar su estructura:

```
ls
ls src/grupo02_robot_kinematics/
    grupo02_robot_kinematics
```

La segunda orden debe mostrar, entre otros archivos, `fk_node.py`, `ik_node.py` y `__init__.py`. De esta manera, los nodos de cinemática quedan disponibles localmente sin necesidad de copiar el código archivo por archivo.

El workspace contiene además los scripts `entorno.sh` e `instalar.sh`. El primero se utiliza para cargar el entorno del proyecto antes de ejecutar los nodos. Si se trata de una instalación nueva y el proyecto requiere instalar dependencias adicionales, se puede ejecutar el instalador incluido en el repositorio:

```
cd ~/grupo2_kuka
chmod +x instalar.sh
./instalar.sh
```

Una vez instaladas las dependencias, se carga el entorno y se compila el workspace:

```
cd ~/grupo2_kuka
source entorno.sh
colcon build --symlink-install
source install/setup.bash
```

Si la compilación termina correctamente, el paquete `grupo02_robot_kinematics` queda disponible para ejecutar los nodos FK e IK. Cada nueva terminal utilizada para trabajar con el proyecto debe cargar nuevamente el entorno:

```
cd ~/grupo2_kuka
source entorno.sh
```

VI-B. Secuencia completa de arranque

Una vez instalado y compilado el workspace, se puede ejecutar la práctica desde el directorio del proyecto. Primero se abre el modelo del robot en RViz2:

```
cd ~/grupo2_kuka
source entorno.sh
ros2 launch grupo02_kuka_kr7_bringup display.launch.py
```

El lanzamiento inicia la visualización del robot y permite utilizar el Joint State Publisher GUI para generar configuraciones articulares. Antes de comenzar una prueba de IK se debe cerrar el GUI, ya que tanto el GUI como `ik_node.py` pueden publicar sobre `/joint_states`.

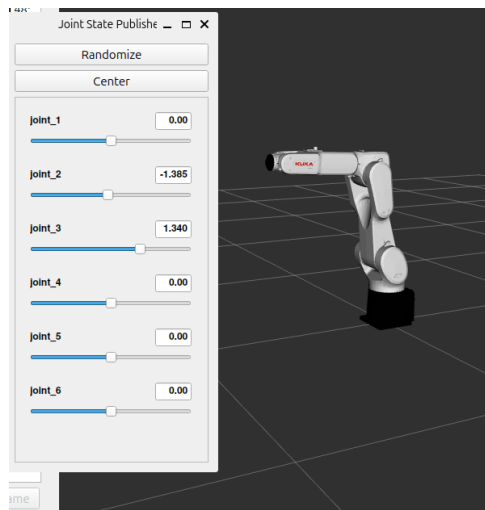


Figura 3. Evidencia de ejecución: Joint State Publisher GUI con las seis articulaciones y el modelo KUKA visualizado en RViz2.

Como comprobación inicial se recomienda ejecutar:

```
ros2 node list
ros2 topic list
ros2 topic echo /joint_states --once
ros2 topic echo /robot_description --once
ros2 run tf2_ros tf2_echo base_link tool0
```

Para compilar el paquete se utilizó:

```
cd ~/grupo2_kuka
source entorno.sh
colcon build --packages-select
grupo02_robot_kinematics --symlink-install
```

Para ejecutar FK:

```
cd ~/grupo2_kuka
source entorno.sh
ros2 run grupo02_robot_kinematics fk_node
```

Para ejecutar IK:

```
cd ~/grupo2_kuka
source entorno.sh
ros2 run grupo02_robot_kinematics ik_node
```

Un objetivo cartesiano se envía mediante:

```
ros2 topic pub /target geometry_msgs/msg/Point \
"{x: 0.400, y: 0.200, z: 0.700}" --once
```

Durante las pruebas de FK se utilizó el Joint State Publisher GUI para generar configuraciones articulares. Para las pruebas de IK se cerró dicho GUI antes de publicar la solución desde `ik_node.py`, evitando dos publishers simultáneos sobre `/joint_states`. El flujo de validación fue

```
/target → ik_node → /joint_states
→ robot_state_publisher → RViz2.
(29)
```

VI-C. Procedimiento reproducible de las pruebas

Prueba de FK. Manteniendo abierto RViz2 y el Joint State Publisher GUI, se mueve una o varias articulaciones hasta obtener una configuración de prueba. En otra terminal se ejecuta:

```
cd ~/grupo2_kuka
source entorno.sh
ros2 run grupo02_robot_kinematics fk_node
```

La posición calculada por FK se compara con la transformación de ROS 2:

```
ros2 run tf2_ros tf2_echo base_link tool0
```

Prueba de IK. Se cierra el Joint State Publisher GUI, se ejecuta `ik_node.py` y se publica el objetivo mediante `/target`. Por ejemplo:

```
cd ~/grupo2_kuka
source entorno.sh
ros2 run grupo02_robot_kinematics ik_node
```

En otra terminal:

```
cd ~/grupo2_kuka
source entorno.sh
ros2 topic pub /target geometry_msgs/msg/Point \
"{x: 0.400, y: 0.200, z: 0.700}" --once
```

La solución se publica en `/joint_states` y se visualiza mediante `robot_state_publisher` y RViz2. Para cada objetivo se verifica posteriormente la posición alcanzada mediante FK y se comprueba que el error sea inferior a la tolerancia de 10^{-3} m.

VII. VALIDACIÓN Y RESULTADOS

VII-A. Validación de la Cinemática Directa

Se compararon los resultados de `fk_node.py` con `tf2_echo base_link tool0`. Las diferencias de posición mostradas corresponden al redondeo de los valores impresos por consola.

Cuadro IV
VALIDACIÓN DE LA CINEMÁTICA DIRECTA MEDIANTE TF

Caso	q [rad]	Validación cartesiana
1	0, -0,0008, -0,0004, 0, 0, 0	$p_{FK} = [0,9769, 0, 0,3879]$ $p_{TF} = [0,977, 0, 0,388]$ $e_p = 1,4 \times 10^{-4}$ m
2	2,8035, -1,573, -1,5713, -1,4685, -0,8199, -2,779	$p_{FK} = [0,3996, 0,1999, 0,7000]$ $p_{TF} = [0,400, 0,200, 0,700]$ $e_p = 4,1 \times 10^{-4}$ m
3	0,4787, -1,9263, 2,1676, 0,5782, 1,1999, 0	$p_{FK} = [0,2997, -0,1997, 0,5999]$ $p_{TF} = [0,300, -0,200, 0,600]$ $e_p = 4,4 \times 10^{-4}$ m

La orientación también coincidió con TF dentro del redondeo de consola. Los cuaterniones reportados por FK y TF fueron, respectivamente, para los tres casos aproximadamente $[-0,0519, 0,7641, -0,0563, 0,6405]$, $[-0,0056, 0,7336, 0,6197, 0,2789]$ y $[0,1433, 0,9496, -0,2788, -0,0041]$.

VII-B. Validación de la Cinemática Inversa

Se realizaron tres pruebas consecutivas. La primera semilla de cada prueba corresponde a la configuración actual del robot en ese momento.

Cuadro V
PRUEBAS EXPERIMENTALES DE LA CINEMÁTICA INVERSA

Caso	Objetivo y configuración	Resultado
1	$p_d = [0,977, 0, 0,388]$ $q_0 = [0, -0,0008, -0,0004, 0, 0, 0]$	$q^* = [-0,0017, -0,073, 0,1086, 0,1533, 0,140, 0]$ 22 iter., $e = 0,00085$ m
2	$p_d = [0,400, 0,200, 0,700]$ $q_0 = [-0,0017, -0,073, 0,1086, 0,1533, 0,140, 0]$	$q^* = [2,8035, -1,573, -1,5713, -1,4685, -0,8199, -2,779]$ 27 iter., $e = 0,00041$ m
3	$p_d = [0,300, -0,200, 0,600]$ $q_0 = [2,8035, -1,573, -1,5713, -1,4685, -0,8199, -2,779]$	$q^* = [0,4787, -1,9263, 2,1676, 0,5782, 1,1999, 0]$ 23 iter., $e = 0,00041$ m

En las tres pruebas la solución final fue publicada porque el error fue inferior a 1 mm. Como comprobación adicional, cada vector q^* se volvió a introducir en FK y la posición calculada coincidió con el objetivo dentro del error reportado.

En la tercera prueba se observó además el beneficio de las múltiples semillas: una de ellas terminó en 300 iteraciones con un error de 0,09854 m y no convergió, mientras que otras semillas sí convergieron. El algoritmo seleccionó la solución de menor error, de 0,00041 m.

VIII. INTEGRACIÓN Y REPOSITORIO

El repositorio utilizado para el proyecto es:

GitHub: ManuelZapanaGutierrez/grupo2_kuka

La rama de trabajo es `main`. El commit de código FK/IK validado durante las pruebas fue `4a93d33`. El repositorio contiene el paquete de cinemática, los archivos necesarios del workspace y el entorno utilizado para ejecutar la práctica.

IX. CONCLUSIONES

- Se implementó una cinemática directa consistente con el modelo utilizado por los nodos FK e IK, empleando la misma tabla DH y la misma transformación fija de referencia.
- La posición y orientación obtenidas mediante FK fueron verificadas contra las transformaciones TF de ROS 2 para tres configuraciones articulares. Las diferencias de posición fueron inferiores a 0,5 mm considerando los valores impresos y su redondeo.
- La cinemática inversa utiliza un Jacobiano posicional numérico por diferencias finitas y una pseudoinversa amortiguada. Los límites articulares, el límite de incremento de 0,20 rad y el criterio de error de 1 mm permiten controlar el proceso iterativo.
- El uso de seis semillas permite encontrar soluciones válidas incluso cuando alguna configuración inicial no converge. En las tres pruebas realizadas se obtuvo una solución con error inferior a 1 mm.
- La integración con ROS 2 permite recibir objetivos mediante `/target`, publicar las soluciones mediante `/joint_states` y comprobar el resultado visualmente mediante `robot_state_publisher`, TF y RViz2.

REFERENCIAS

- [1] KUKA, "Product Guide KUKA China," KUKA Roboter GmbH, 2025. [En línea]. Disponible en: [kuka.com product guide](https://www.kuka.com/product-guide).
- [2] KUKA Roboter GmbH, "KR AGILUS sixx with W wrist: Specification," Augsburg, Alemania, 2023.
- [3] ROS-Industrial, "kuka_robot_descriptions (ROS 2)," GitHub repository, 2024. [En línea]. Disponible en: github.com/kroshu/kuka_robot_descriptions
- [4] M. W. Spong, S. Hutchinson, y M. Vidyasagar, *Robot Modeling and Control*, 2da ed. Hoboken, NJ: John Wiley & Sons, 2020.